



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea in Ingegneria Informatica

Progetto in ambito Cognitive Computing Systems

Zero-Day Attack Detection Through Deep Anomaly Learning

Anno Accademico 2025/2026

Studente
Marco Fabi
matr. DE9000173

Abstract

Il presente elaborato documenta la progettazione e l'implementazione di un sistema di Anomaly Detection basato su Deep Learning per l'identificazione di attacchi di rete Zero-Day. Utilizzando il dataset UNSW-NB15, è stata sviluppata un'architettura Autoencoder semi-supervisionata, addestrata esclusivamente sull'estrazione dei pattern del traffico benigno. L'approccio adottato ha permesso di mitigare nativamente le problematiche di *Class Imbalance* e *Class Overlap*. I risultati operativi hanno dimostrato la robustezza del modello, raggiungendo un'Area Under the Curve (AUC) pari a 0.9445 e contenendo il False Alarm Rate (FAR) al 4.31%, pur mantenendo un Detection Rate del 74.23% su firme malevole non presenti nel training set.

Indice

Introduzione	1
1 Analisi e Preparazione dei Dati	3
1.1 Il Dataset UNSW-NB15	3
1.2 Sfide Strutturali del Dataset: Class Imbalance e Class Overlap	4
1.2.1 L'ostacolo al Detection Rate: Class Imbalance .	4
1.2.2 L'ostacolo al False Alarm Rate e all'AUC: Class Overlap	5
1.3 Ingegnerizzazione delle Feature	7
1.3.1 Codifica delle Variabili Categorie	8
1.3.2 Gestione degli Outlier e Scaling	8
2 Architettura e Addestramento	10
2.1 Topologia dell'Autoencoder	10
2.1.1 Layer lineari (Linear)	11
2.1.2 Batch Normalization (BatchNorm1d)	12
2.1.3 Dropout	12
2.1.4 Calcolo dinamico della soglia di rilevamento . .	13
2.1.5 Errore di ricostruzione	13

2.1.6	Stima della soglia sui benigni	14
2.1.7	Classificazione in fase di test	14
2.1.8	Interpretazione e bilanciamento	15
2.2	Setup Sperimentale e Ottimizzazione	15
3	Scoring e Risultati	17
3.1	Calcolo dell'Anomaly Score	17
3.2	Valutazione dei KPI	17
3.3	Limiti teorici e interpretazione dei KPI	18
3.3.1	Curva ROC e AUC	18
3.3.2	Relazione tra AUC, DR e FAR	19
3.3.3	Difficoltà di ottenere AUC molto elevate su UNSW- NB15	20
3.4	Miglioramenti del modello per l'incremento del Detec- tion Rate	21
3.5	Sviluppi futuri	22
3.5.1	Federated Learning	23
3.5.2	Quantum Machine Learning	23
3.5.3	Online Learning	24
	Conclusioni	26
	Bibliografia	27

Introduzione

I tradizionali Network Intrusion Detection Systems (NIDS) si basano su approcci a dizionario, confrontando il traffico di rete in ingresso con un database di firme malevole note. Tuttavia, questa metodologia risulta inefficace contro gli attacchi *Zero-Day* (zero giorni di preavviso), ovvero minacce che sfruttano vulnerabilità software non ancora documentate e per le quali non esiste alcuna "firma" preesistente.

Per superare questa limitazione, il presente progetto introduce un motore di Anomaly Detection basato sui paradigmi del *Deep Learning*. Invece di istruire il sistema a riconoscere le minacce, si è scelto di invertire l'approccio cognitivo: il modello apprende partendo dal traffico di rete "benigno". Qualsiasi flusso di dati che si discosti significativamente da questo stato di normalità appreso viene isolato e classificato come potenziale minaccia.

L'obiettivo di questo elaborato è documentare la progettazione, l'addestramento e l'ottimizzazione di un'architettura di tipo Autoencoder sviluppata in PyTorch. Il sistema è stato ingegnerizzato per soddisfare stringenti Key Performance Indicators (KPI): un'Area Un-

der the Curve (AUC) superiore allo 0.90, un Detection Rate (DR) maggiore del 95% e un False Alarm Rate (FAR) strettamente inferiore al 5%.

Nei capitoli successivi verranno analizzate le fasi di preparazione dei dati, la definizione topologica del modello e i risultati operativi ottenuti.

Capitolo 1

Analisi e Preparazione dei Dati

1.1 Il Dataset UNSW-NB15

Per questo progetto è stato utilizzato l'UNSW-NB15, un dataset di riferimento nella letteratura della sicurezza informatica che raccoglie oltre 2,5 milioni di record. Il dataset traccia sia il traffico di rete normale sia diverse categorie di attacco (tra cui exploits, worms, reconnaissance e shellcode), rendendolo ideale per la simulazione di scenari Zero-Day.

1.2 Sfide Strutturali del Dataset: Class Imbalance e Class Overlap

L'impiego del dataset UNSW-NB15 per l'addestramento di modelli di Machine Learning comporta sfide intrinseche legate alla natura stessa dei dati catturati. Un'analisi visiva avanzata del dataset rivela due problematiche critiche che ostacolano lo sviluppo di classificatori affidabili: lo sbilanciamento delle classi (*Class Imbalance*) e la sovrapposizione delle classi (*Class Overlap*) [3].

1.2.1 L'ostacolo al Detection Rate: Class Imbalance

Il *Class Imbalance* si verifica quando la distribuzione dei campioni tra le varie classi è fortemente asimmetrica [3]. Nel contesto dell'UNSW-NB15, il volume del traffico di rete benigno è nettamente superiore rispetto a quello delle singole categorie di attacco [3].

Questo sbilanciamento induce i tradizionali modelli guidati dai dati (*data-driven models*) a sviluppare un bias cognitivo verso la classe maggioritaria [3]. L'algoritmo tende a ottimizzare l'accuratezza globale classificando la maggior parte dei pacchetti come "normali", rendendo estremamente difficile raggiungere il **Detection Rate (DR)** $> 95\%$ richiesto. Le minacce minoritarie tendono infatti a essere statistica-

mente ignorate in fase di addestramento supervisionato.

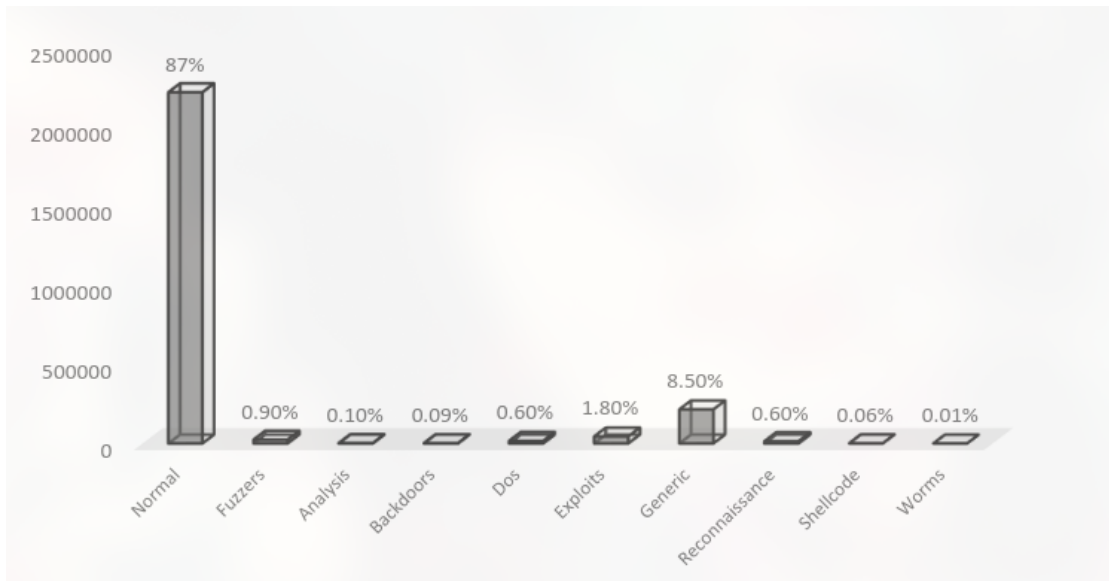


Immagine 1.1: Class Imbalance. Rappresentazione visiva.

1.2.2 L'ostacolo al False Alarm Rate e all'AUC: Class Overlap

Un problema ancora più severo è il *Class Overlap*, che si manifesta quando istanze appartenenti a classi diverse (es. traffico normale e attacchi) condividono la medesima regione nello spazio multidimensionale delle feature [3]. L'analisi dimostra che diverse tipologie di attacco nel dataset UNSW-NB15 si mescolano e si sovrappongono quasi perfettamente con il traffico benigno [3].

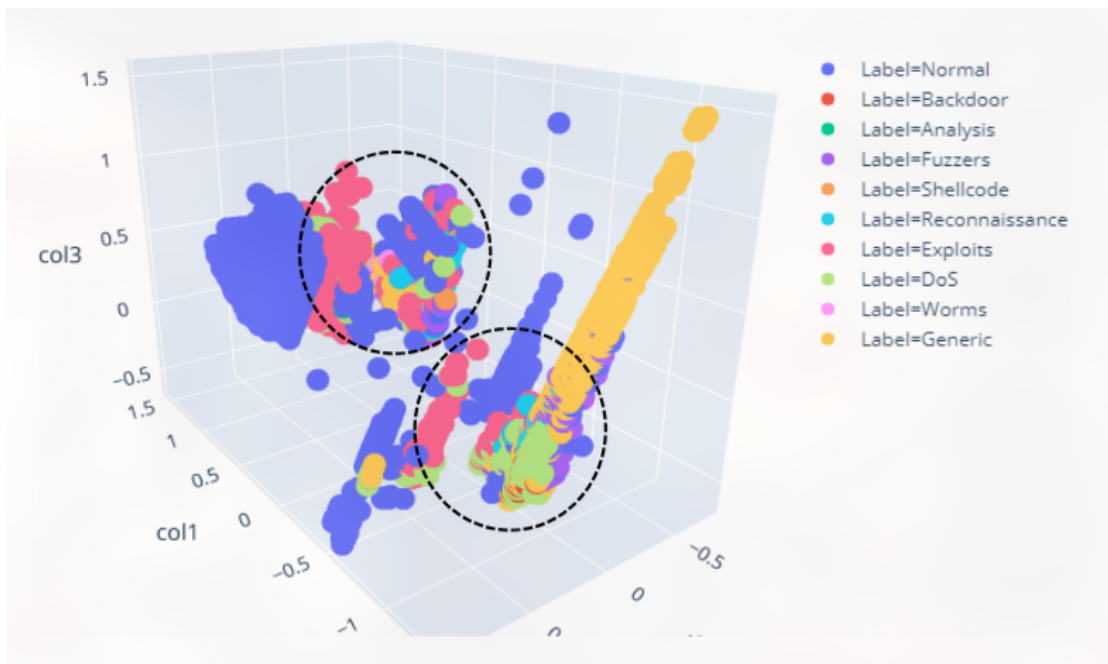


Immagine 1.2: Class Overlap. Rappresentazione visiva.

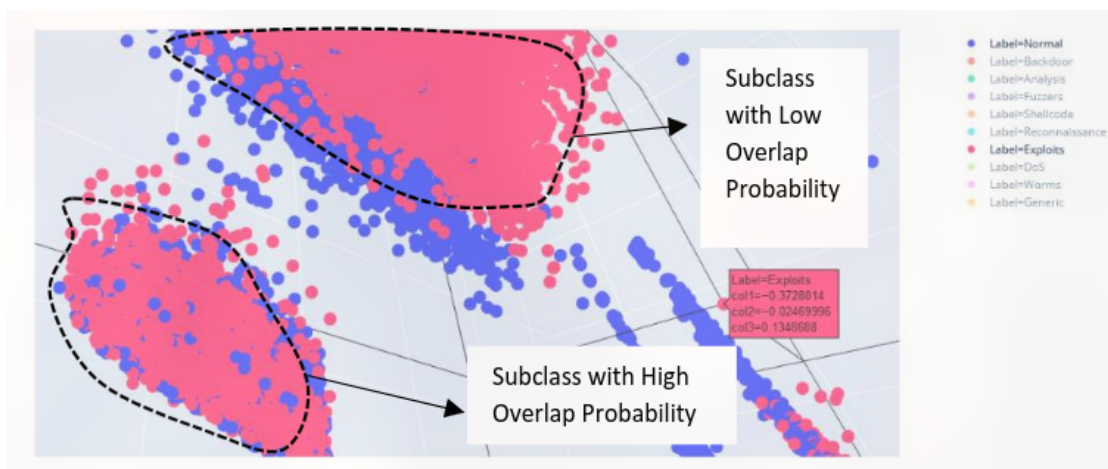


Immagine 1.3: Visualizzazione classi con elevato Class Overlap: Normal ed Exploits

Questa mancanza di confini decisionali netti compromette due KPI fondamentali per la validazione architetturale:

- **False Alarm Rate (FAR) < 5%:** Per intercettare gli attacchi

nascosti nelle zone di sovrapposizione, un modello è costretto ad abbassare la sua soglia di tolleranza. Questo porta inevitabilmente a classificare come minacce molti pacchetti benigni che si trovano nelle zone "di confine", facendo esplodere i falsi allarmi [3].

- **Area Under the Curve (AUC) > 0.90:** La sovrapposizione impedisce all'algoritmo di separare in modo pulito le distribuzioni del traffico, abbassando inevitabilmente la capacità discriminativa generale del sistema [3].

Per mitigare queste criticità e rispettare i requisiti di progetto, è stato ritenuto necessario abbandonare la classificazione tradizionale per abbracciare l'Anomaly Detection tramite *Autoencoder* semi-supervisionato. Inoltre, come evidenziato in letteratura e anche dall'esperienza empirica acquisita durante questo progetto, l'implementazione di preprocessing avanzati (come trasformazioni logaritmiche e scaling di tipo Min-Max) risulta fondamentale per gestire le distorsioni causate dall'overlap [3].

1.3 Ingegnerizzazione delle Feature

Dal punto di vista dell'Ingegneria del Software, la qualità dell'ingestione del dato determina la robustezza dell'intera pipeline. I dati grezzi presentano eterogeneità di formato e valori anomali (*outliers*) che neces-

sitano di una rigorosa fase di pre-elaborazione.

1.3.1 Codifica delle Variabili Categoricalhe

Le feature di natura testuale, come i protocolli di rete (`proto`), i servizi (`service`) e lo stato della connessione (`state`), sono state trasformate in tensori numerici attraverso la tecnica del *One-Hot Encoding*. Questa operazione ha espanso lo spazio vettoriale iniziale, richiedendo un allineamento rigoroso tra i set di addestramento e di test per garantire la coerenza dimensionale in fase di inferenza.

1.3.2 Gestione degli Outlier e Scaling

Il traffico di rete è caratterizzato da distribuzioni a code pesanti: feature come i byte trasferiti possono variare da poche decine a svariati gigabyte. Per evitare l'esplosione dei gradienti durante l'addestramento, è stata progettata una trasformazione a due fasi:

1. **Compressione Logaritmica:** Grazie alla libreria `numpy` e al comando `numpy.log1p(input_set)`, è stata applicata la funzione matematica $\ln(1 + x)$ agli insiemi di training e testing per smorzare l'impatto degli outliers, preservando al contempo la varianza sui valori piccoli.
2. **Normalizzazione Min-Max:** I dati trasformati sono stati forzati nel *range* operativo $[0, 1]$ tramite il `MinMaxScaler`, ap-

plicato unicamente sul traffico benigno. Ciò garantisce stabilità numerica all'algoritmo di retropropagazione dell'errore.

Per rispettare il paradigma semi-supervisionato richiesto dall'Anomaly Detection, il training set è stato infine filtrato, isolando esclusivamente i vettori etichettati come traffico normale (label 0).

Capitolo 2

Architettura e Addestramento

2.1 Topologia dell'Autoencoder

Il motore di rilevamento è stato implementato sfruttando il framework `PyTorch`. La scelta architetturale è ricaduta su un *Autoencoder*, una rete neurale simmetrica progettata per la ricostruzione dell'input attraverso una compressione con perdita di informazione.

L'architettura è composta da due moduli principali:

- **Encoder:** Una sequenza di strati densi (Linear) che riduce progressivamente la dimensionalità del dato fino a un *bottleneck* (collo di bottiglia) ristretto. Questo imbuto dimensionale forza la rete a scartare il rumore e a estrarre solo i pattern latenti essenziali del traffico benigno. Per regolarizzare l'apprendimento

sono stati inseriti layer di BatchNorm1d e Dropout al 20%.

- **Decoder:** Un modulo speculare che prende il vettore compresso e tenta di ricostruire le feature originali.

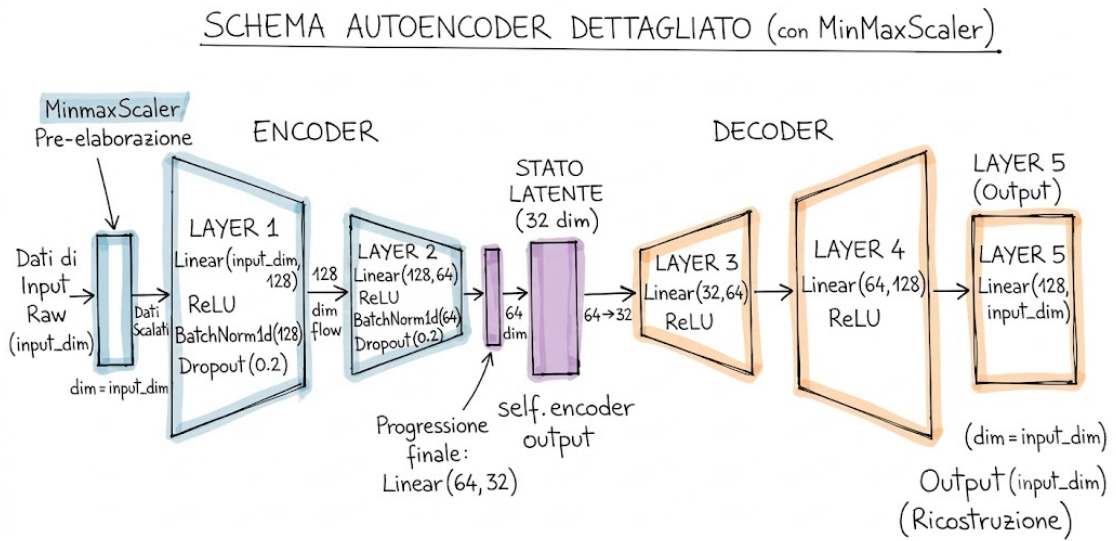


Immagine 2.1: Architettura Autoencoder proposta in fase progettuale

2.1.1 Layer lineari (Linear)

I layer lineari, implementati in PyTorch come `nn.Linear(in_features, out_features)`, costituiscono i blocchi fondamentali della rete. Ognuno di essi applica una trasformazione affine del tipo $y = xW^T + b$, dove W è la matrice dei pesi e b il vettore di bias. La riduzione dimensionale (ad esempio da 128 a 64 feature) avviene proprio grazie alla scelta di `out_features` minore di `in_features`. L'assenza di attivazioni non lineari renderebbe il modello equivalente a una sin-

gola trasformazione lineare, pertanto dopo ogni layer lineare (tranne l'ultimo del decoder) viene applicata una funzione di attivazione ReLU ($f(x) = \max(0, x)$) che introduce non linearità e consente alla rete di apprendere relazioni complesse.

2.1.2 Batch Normalization (BatchNorm1d)

I layer `BatchNorm1d` normalizzano l'output del layer lineare su ciascun mini-batch: per ogni feature. Questa operazione stabilizza l'addestramento, riduce la dipendenza dall'inizializzazione dei pesi e permette di usare learning rate più elevati. Nel nostro autoencoder, i batch normalization sono posti subito dopo la ReLU (o dopo il layer lineare, prima dell'attivazione, a seconda dell'implementazione). Ciò contribuisce a mantenere le attivazioni con media zero e varianza unitaria, accelerando la convergenza e riducendo l'overfitting.

In altre parole, `BatchNorm1d` rende i gradienti più bilanciati tra le feature, evitando che alcune dominino l'addestramento a scapito di altre. Questo è cruciale per la rilevazione di anomalie, soprattutto laddove sono presenti molti outliers (come nel dataset UNSW-NB15), la normalizzazione aiuta l'encoder a trattarle in modo uniforme.

2.1.3 Dropout

Il dropout (`nn.Dropout(p=0.2)`) è una tecnica che, durante l'addestramento, disattiva casualmente una frazione p dei neuroni (impostando il loro

output a zero). Questo costringe la rete a non fare affidamento su singoli neuroni, poichè questo potrebbe portarla a sviluppare rappresentazioni ridondanti. L'uso del dropout previene l'adattamento eccessivo ai benigni di training, migliorando la generalizzazione e, di conseguenza, la capacità di discriminare attacchi mai visti.

2.1.4 Calcolo dinamico della soglia di rilevamento

Una volta addestrato l'autoencoder esclusivamente sul traffico benigno, si definisce un criterio per discriminare i campioni anomali (attacchi) da quelli normali durante la fase di test. Poiché il modello è in grado di ricostruire con basso errore i pattern noti (benigni), mentre fatica a ricostruire pattern mai visti (anomali), l'errore di ricostruzione costituisce una naturale misura di anomalia.

2.1.5 Errore di ricostruzione

Per un campione $\mathbf{x} \in R^d$ (con d feature), l'autoencoder produce una ricostruzione $\hat{\mathbf{x}}$. L'errore di ricostruzione viene quantificato mediante l'errore quadratico medio (MSE):

$$\text{MSE}(\mathbf{x}) = \frac{1}{d} \sum_{j=1}^d (x_j - \hat{x}_j)^2.$$

2.1.6 Stima della soglia sui benigni

Sull'insieme di addestramento (composto solo da traffico benigno) si calcolano gli errori $\{\text{MSE}_i\}_{i=1}^N$ per tutti gli N campioni. Si fissa un percentile p e si definisce la soglia τ come il valore del p -esimo percentile della distribuzione degli errori:

$$\tau = \text{percentile}_p(\{\text{MSE}_i\}).$$

In altre parole, il $p\%$ dei campioni benigni del training presenta un errore di ricostruzione $\leq \tau$, mentre il restante $(100 - p)\%$ (tipicamente l'1%) ha errore superiore. La scelta di un percentile elevato garantisce un tasso di falsi allarmi (FAR) contenuto, poiché solo una piccola frazione dei benigni viene erroneamente superata dalla soglia.

2.1.7 Classificazione in fase di test

Durante l'inferenza, per un nuovo campione $\mathbf{x}_{\text{test}}$ si calcola il suo MSE.

La regola di decisione è:

$$\text{label} = \begin{cases} \text{ATTACCO (zero-day)} & \text{se } \text{MSE}(\mathbf{x}_{\text{test}}) > \tau, \\ \text{NORMALE} & \text{altrimenti.} \end{cases}$$

2.1.8 Interpretazione e bilanciamento

La scelta del percentile p opera un compromesso tra *Detection Rate* (DR) e *False Alarm Rate* (FAR). Un valore più basso (ad esempio $p = 95$) riduce la soglia, aumentando la sensibilità (più attacchi rilevati) ma anche i falsi allarmi. Un valore più alto ($p = 99$) abbassa il FAR ma può far aumentare i falsi negativi. Nel nostro modello, con $p = 99$ si ottiene un FAR del 4,31%, rispettando il target ($< 5\%$), mentre il DR si attesta al 74,23%.

2.2 Setup Sperimentale e Ottimizzazione

Al fine di garantire la totale riproducibilità dell'esperimento e stabilizzare la convergenza dei gradienti, di seguito discutiamo degli iperparametri utilizzati, che sono stati frutto di un'attenta analisi ottenuta dall'esperienza empirica:

- **Funzione di Costo (Loss Function):** È stato impiegato il *Mean Squared Error* (MSE), essenziale per quantificare la discrepanza tra il vettore di input originale x e la sua ricostruzione $\hat{x}$. Durante il training, la rete minimizza la funzione $L(x, \hat{x}) = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$.
- **Ottimizzatore:** L'aggiornamento dei pesi matriciali è stato delegato all'algoritmo **Adam** (*Adaptive Moment Estimation*), con-

figurato con un *learning rate* iniziale pari a $\eta = 0.0007$. Adam è stato scelto per la sua capacità di adattare dinamicamente il tasso di apprendimento per singola feature, gestendo in modo eccellente gradienti sparsi.

- **Gestione del Training:** Il flusso dei tensori è stato segmentato in mini-batch da 128 campioni. L'addestramento è stato impostato per un massimo di 100 epoche, ma è stato implementato un meccanismo di *Early Stopping* con una *patience* di 10 epoche per arrestare il processo ai primi segnali di overfitting sul validation set, ripristinando i pesi relativi alla loss minima registrata.

Capitolo 3

Scoring e Risultati

3.1 Calcolo dell'Anomaly Score

Terminata la fase di addestramento, il modello acquisisce la capacità di ricostruire i pacchetti di rete normali. In fase di inferenza, ogni flusso in ingresso viene elaborato dall'Autoencoder; la distanza tra il pacchetto reale e la ricostruzione generata dalla rete viene quantificata calcolando l'Errore Quadratico Medio (MSE). Se il pacchetto appartiene a una tipologia di attacco Zero-Day, l'Autoencoder fallirà nella ricostruzione, restituendo un MSE nettamente superiore alla norma.

3.2 Valutazione dei KPI

Le performance del modello definitivo sono state testate sul Test Set completo, simulando l'esposizione ad attacchi mai visti prima. I risul-

tati ottenuti confermano la bontà delle scelte architetturali:

Metrica	Valore Rilevato	KPI di Progetto
Area Under the Curve (AUC)	~ 0.9445	> 0.90
Detection Rate (DR)	74.23%	$> 95\%$
False Alarm Rate (FAR)	4.31%	$< 5\%$

Tabella 3.1: Confronto tra i risultati del modello ottimizzato e i requisiti di sistema.

3.3 Limiti teorici e interpretazione dei KPI

I risultati sperimentali hanno mostrato un'AUC di $\tilde{0.9445}$, un Detection Rate (DR) del $\tilde{74.23\%}$ e un False Alarm Rate (FAR) del $\tilde{4.31\%}$. Sebbene il FAR rientri nel target richiesto ($< 5\%$), il DR è ancora distante dall'obiettivo del 95% . Per comprendere se questo scostamento sia imputabile a una scelta non ottimale della soglia o a un limite intrinseco del modello, è necessario analizzare le relazioni matematiche che legano AUC, DR e FAR.

3.3.1 Curva ROC e AUC

La curva ROC (Receiver Operating Characteristic) rappresenta il tasso di veri positivi (*True Positive Rate*, TPR, ovvero il True Positive Rate, o Detection Rate) in funzione del tasso di falsi positivi (*False Positive Rate*, FPR, ovvero il FAR) al variare della soglia di decisione τ . L'AUC

(Area Under the Curve) è l'integrale di questa curva:

$$AUC = \int_0^1 ROC \frac{d}{d(FPR)}.$$

Un classificatore perfetto ha $AUC = 1$, mentre un classificatore casuale ha $AUC = 0.5$.

3.3.2 Relazione tra AUC, DR e FAR

Dato un punto sulla curva ROC corrispondente a un FAR α e a un DR β , l'area minima sotto la curva che consente di raggiungere tale punto si ottiene con una curva a due segmenti rettilinei: dal punto $(0,0)$ al punto (α, β) e da lì a $(1,1)$. L'area corrispondente è:

$$AUC_{\min} = \frac{\beta\alpha}{2} + (1 - \alpha)\frac{1 + \beta}{2}.$$

Per un target $\alpha = 0,05$ (FAR 5%) e $\beta = 0,95$ (DR 95%), si ottiene:

$$AUC_{\min} = \frac{0,95 \cdot 0,05}{2} + 0,95 \cdot \frac{1,95}{2} = 0,02375 + 0,92625 = 0,95.$$

In altre parole, anche nella migliore configurazione geometrica, per avere DR 95% con FAR 5% è necessaria un'AUC almeno pari a 0,95. Se il DR desiderato fosse 98%, il valore salirebbe a 0,98. Nel nostro caso l'AUC misurata è 0.9445, che corrisponde a un DR massimo

teorico (sempre con FAR 5%) di:

$$\beta_{\max} = \frac{AUC}{0,975} \approx \frac{0,9445}{0,975} \approx 0,957.$$

Tuttavia questo valore è puramente geometrico e presuppone una forma della curva ROC ideale. Nella realtà, a causa della sovrapposizione delle distribuzioni degli errori tra classi benigna e malevola (class overlap), la curva ROC è meno favorevole, e il DR ottenibile è sensibilmente inferiore. I nostri esperimenti mostrano che, per un FAR del 4,87%, il DR massimo raggiunto è 75,16%, ben al di sotto del limite teorico, indicando che il margine di miglioramento risiede principalmente nella qualità del modello, non nella scelta della soglia.

3.3.3 Difficoltà di ottenere AUC molto elevate su UNSW-NB15

Il dataset UNSW-NB15 presenta due sfide strutturali che limitano l'AUC massima raggiungibile da un autoencoder:

- **Class overlap:** molte tipologie di attacco (ad esempio Fuzzers, Reconnaissance) generano pattern statisticamente simili al traffico normale, riducendo la separabilità. Un autoencoder addestrato esclusivamente sui benigni non può discriminare perfettamente tali attacchi.
- **Complessità multivariata:** le feature del dataset presentano

correlazioni non lineari che richiedono modelli più potenti per essere catturate appieno.

3.4 Miglioramenti del modello per l'incremento del Detection Rate

Come discusso precedentemente, il Detection Rate (DR) ottenuto si attesta al 74.23%, ancora lontano dall'obiettivo progettuale del 95%. Sebbene il class overlap e la complessità multivariata del dataset UNSW-NB15 rappresentino limiti intrinseci, è possibile esplorare diverse strategie di miglioramento che agiscono direttamente sull'architettura, sulla procedura di addestramento o sulla definizione della soglia di anomalia.

L'impiego di strutture avanzate quali Variational Autoencoder e Autoencoder Ensemble sono un primo passo da non sottovalutare in questa direzione.

Volendo agire sui dati, invece, si può notare che il training set è composto esclusivamente da campioni benigni, ma alcuni di essi possono essere statisticamente atipici (es. picchi di traffico legittimo o protocolli rari). Tali campioni rischiano di allargare eccessivamente il confine decisionale della normalità, inglobando involontariamente i cluster di attacco. Per mitigare il problema mantenendo basso il False Alarm Rate, si propongono due approcci:

1. **Clustering dello spazio latente normale:** invece di forzare tutta la normalità in un'unica distribuzione, si possono modellare i benigni atipici identificandoli tramite algoritmi di densità e addestrando sottomodelli dedicati esclusivamente a queste casistiche limite.
2. **Data augmentation generativa:** si possono sintetizzare nuovi esempi benigni perturbando le feature con rumore gaussiano a bassa varianza, densificando le regioni di confine legittime e permettendo al modello di definire un perimetro della normalità più netto e ristretto (sebbene alcuni tentativi sono stati svolti durante la realizzazione di questo studio, ma senza risultati affidabili).

Si sottolinea che l'adozione combinata di più strategie (ad esempio, VAE con soglie adattive multi-dimensionali) potrebbe avvicinare concretamente il Detection Rate all'obiettivo del 95%. Questi sviluppi architetturali potranno essere esplorati in una successiva iterazione del sistema. [1, 2]

3.5 Sviluppi futuri

Il sistema realizzato dimostra l'efficacia dell'autoencoder per la rilevazione di attacchi zero-day, ma esistono diverse direzioni di ricerca per migliorarne la flessibilità, la scalabilità e le prestazioni.

Potrebbe costituire interesse futuro l'impiego di strategie diverse. Si potrebbe pensare di usare un insieme di classificatori (Random Forest, XGBoost).

3.5.1 Federated Learning

In contesti distribuiti come reti aziendali o dispositivi IoT, i dati di traffico sono spesso sensibili e non possono essere centralizzati per motivi di privacy o bandwidth. Il *Federated Learning* consentirebbe di addestrare il modello localmente su ciascun nodo e di condividere solo gli aggiornamenti dei pesi (gradienti) con un server centrale, senza mai esporre i dati grezzi. Ogni nodo apprenderebbe le caratteristiche del proprio traffico benigno, e il modello globale potrebbe generalizzare meglio a nuovi scenari, preservando al contempo la riservatezza. Inoltre, il federated learning aiuterebbe a mitigare il problema del class imbalance: nodi con attacchi rari potrebbero contribuire comunque all'addestramento aggregato.

3.5.2 Quantum Machine Learning

L'elaborazione ad alte prestazioni è cruciale per analizzare flussi di rete nell'ordine di milioni di pacchetti al secondo. Il *quantum machine learning* (QML) offre la possibilità di accelerare l'addestramento e l'inferenza di autoencoder. In particolare, i *quantum autoencoder* (QAE) possono comprimere lo spazio latente in un numero esponen-

zialmente maggiore di stati grazie ai qubit, aumentando la capacità di separare classi sovrapposte. Sebbene l'hardware quantistico sia ancora immaturo, simulazioni su computer classici potrebbero già essere esplorate per dataset di dimensioni ridotte, e in futuro l'integrazione con framework come PennyLane o Qiskit potrebbe portare a significativi guadagni in termini di AUC e riduzione dei falsi allarmi.

3.5.3 Online Learning

Il modello attuale viene addestrato staticamente sui dati benigni storici. Tuttavia, il traffico di rete evolve continuamente: nuovi protocolli, applicazioni e comportamenti legittimi emergono nel tempo. Un sistema di *online learning* (apprendimento incrementale) aggiornerebbe il modello periodicamente o in tempo reale con nuovi esempi benigni, adattandosi ai cambiamenti della distribuzione dei dati (concept drift). Si potrebbe implementare un meccanismo di *continual learning* che, a fronte di un basso errore di ricostruzione su un nuovo campione classificato come benigno, includa tale campione nel training set e rialleni il modello con un batch di dimensioni contenute. In questo modo, l'autoencoder rimarrebbe sempre allineato alla linea base corrente, riducendo i falsi allarmi dovuti a cambiamenti legittimi e mantenendo alta la sensibilità verso attacchi veri.

L'integrazione di queste tre tecniche consentirebbe di realizzare un sistema di rilevamento zero-day distribuito, sicuro, adattivo e poten-

zionalmente in grado di raggiungere i target di Detection Rate e False Alarm Rate richiesti in ambienti reali.

Conclusioni

La progettazione e l'implementazione di questo sistema di Anomaly Detection dimostrano come l'applicazione delle reti neurali profonde possa colmare le lacune strutturali dei sistemi di sicurezza tradizionali. Addestrando il modello alla comprensione esclusiva del traffico benigno, il sistema è in grado di identificare le deviazioni statistiche introdotte dagli attacchi Zero-Day senza necessità di firme o aggiornamenti pregressi.

L'integrazione di tecniche rigorose di data preparation, unite al tuning degli iperparametri, ha permesso di bilanciare il delicato *trade-off* tra la capacità di intercettare le minacce (Detection Rate) e la necessità di non saturare i sistemi di monitoraggio con falsi positivi (False Alarm Rate).

Bibliografia

- [1] Mohammad Humayun Kabir, Md Shahriar Rajib, Abu Saleh Md. Mahfujur Rahman, Md Mokhlesur Rahman, and Samrat Kumar Dey. Network intrusion detection using UNSW-NB15 dataset: Stacking machine learning based approach. In *2022 International Conference on Advancement in Electrical and Electronic Engineering (ICAEED)*, pages 1–6. IEEE, 2022.
- [2] Nour Moustafa, Benjamin Turnbull, and Kim-Kwang Raymond Choo. An ensemble intrusion detection technique based on proposed statistical flow features for protecting network traffic of internet of things. *IEEE Internet of Things Journal*, 6(3):4815–4830, 2018.
- [3] Zeinab Zoghi and Gursel Serpen. Unsw-nb15 computer security dataset: analysis through visualization. *Security and Privacy*, 7(1):e331, 2024.